

CyberSec Nexus — Complete Project Explanation

1. Project Introduction

What is CyberSec Nexus?

CyberSec Nexus is a web-based cybersecurity platform built using modern AWT (Advanced Web Technology) technologies.

The platform combines multiple cybersecurity-related functionalities into one dashboard-like system.

It allows users to:

- Register/Login securely
 - Analyze suspicious URLs, emails, and IPs
 - Detect attacks inside server logs
 - Upload and manage security reports
 - Monitor activities through an analyst portal
 - View live cybersecurity news
-

2. Main Goal of the Project

The actual academic goal of this project was:

✓ To implement multiple AWT technologies together in one working application.

Instead of making a simple CRUD app, this project combines:

- Frontend technologies
- Backend technologies
- Database integration
- APIs
- Authentication
- File handling
- Real-time data fetching
- Role-based access
- Cybersecurity concepts

into one single system.

3. Real-Life Analogy

Think of CyberSec Nexus like a:

“Mini Cybersecurity Control Center”

Different departments do different jobs:

Module	Real-Life Analogy
Login System	Security gate
Dashboard	Main control room
ThreatLens	VirusTotal / URL scanner
LogSentry	CCTV footage analyzer
SecureReports	Secure document locker
Analyst Portal	Admin intelligence room

4. Complete Project Architecture

The project has 3 main parts:

Frontend (React)

↓

Backend API Server (Node + Express)

↓

Database (MySQL)

5. Frontend Explanation

What is Frontend?

Frontend means:

👉 Everything the user can SEE and CLICK in the browser.

Example:

- Buttons
- Forms
- Dashboard
- Cards

- Navigation
- Colors
- Animations

Frontend Technology Used

Technology	Purpose
React.js	Build UI
React Router	DOM Navigation
Tailwind CSS	Styling
Axios	API requests
Vite	Fast frontend server
localStorage	Session storage

6. React.js Explanation

What is React?

React is a JavaScript library used for building web interfaces.

Instead of making one giant HTML file, React divides the UI into reusable parts called:

Components

Example:

- Navbar component
- Sidebar component
- Dashboard component
- ThreatLens component

Like LEGO blocks.

7. React Router DOM

Purpose

Handles page navigation without refreshing the browser.

Example routes:

/dashboard

/threatlens

/logsentry

/secure-reports

/analyst-portal

Without React Router:

- Every click reloads the entire website

With React Router:

- Navigation feels instant
-

8. Tailwind CSS

Purpose

Used for styling the UI.

Instead of writing huge CSS files:

background-color: blue;

padding: 20px;

Tailwind allows:

className="bg-blue-500 p-5"

This made your UI:

- Faster to build
 - Modern looking
 - Responsive
-

9. Axios

Purpose

Frontend and backend communication.

Example:

```
axios.post("/api/analyze")
```

Frontend sends request →

Backend processes →

Frontend receives response

Like a delivery service between browser and server.

10. localStorage

Purpose

Stores login session in browser memory.

Example values stored:

isLoggedIn = true

username = "Shravani"

role = "admin"

Used for:

- Keeping user logged in
 - Showing username
 - Admin access checking
-

11. Backend Explanation

What is Backend?

Backend means:

👉 Everything working behind the scenes.

Handles:

- Database operations
 - Login verification
 - Threat analysis
 - File uploads
 - APIs
 - Security logic
-

Backend Technologies Used

Technology Purpose

Node.js	Server runtime
---------	----------------

Technology Purpose

Express.js	API framework
MySQL	Database
bcryptjs	Password hashing
Multer	File upload
Nodemailer	Email alerts
CORS	Frontend-backend connection

12. Node.js

Normally JavaScript only runs in browsers.

Node.js allows JavaScript to run as a SERVER.

Your backend entire logic runs on Node.js.

13. Express.js

Express organizes backend routes.

Example:

```
app.use("/api/auth", authRoutes)
```

This means:

All login/register requests go to auth routes.

Express acts like:

Traffic Police for APIs

14. MySQL Database

Purpose

Permanent storage.

Tables Used

Table	Stores
users	User data

Table	Stores
reports	Uploaded files
audit_logs	Activity logs

Example

When user registers:

Name → users table

Email → users table

Password hash → users table

15. bcryptjs

Purpose

Encrypt passwords.

Password:

hello123

becomes:

\$2a\$10\$hKJ82...

Even database admins cannot read original password.

16. Multer

Purpose

Handles file uploads.

Used in:

SecureReports Module

When user uploads:

- PDF
- TXT
- DOC

Multer:

- Receives file

- Saves into uploads folder
 - Generates unique filename
-

17. Nodemailer

Purpose

Automatic email alerts.

When new user registers:

- Email sent to all 3 admins

This demonstrates:

Email service integration

18. CORS

Frontend runs on:

localhost:5173

Backend runs on:

localhost:5000

Browser blocks communication by default.

CORS allows:

Frontend ↔ Backend communication

19. Modules Explanation

Module 1 — Authentication System

Features

- Register
- Login
- Logout
- Session handling

Working

Register

1. User enters details

2. Password hashed using bcrypt
3. Saved into database
4. Admin email alert triggered

Login

1. User enters credentials
2. Backend compares hashed password
3. If correct:
 - localStorage updated
 - user redirected to dashboard

Module 2 — Dashboard

Purpose

Main overview screen.

Shows:

- Total users
- Reports count
- Threat scans
- Log checks
- System status

These values come LIVE from MySQL database.

Module 3 — ThreatLens

Purpose

Threat analysis engine.

Analyzes:

- URLs
- Emails
- IP addresses

What It Detects

Detection	Example
HTTP usage	http://example.com
Fake domains	paypal.com
Typosquatting	amaz0n.com
Suspicious TLDs	.ru, .xyz
IP-based URLs	192.168.1.1/login
Email spoofing	support@paypa1.com

Important Reality

This is:

Rule-Based Detection

NOT:

- AI
- Machine Learning
- Real malware scanning

The system checks:

- Patterns
- Keywords
- Suspicious structures

That's why some real fake emails may still show LOW risk.

That is NORMAL.

If Viva Asks:

"Why doesn't it detect every real phishing email?"

Answer:

"Our system uses heuristic and pattern-based analysis, not real-time global threat intelligence or machine learning datasets. It demonstrates cybersecurity detection concepts for educational purposes."

That answer is technically correct.

Module 4 — LogSentry

Purpose

Server log analyzer.

Reads logs line-by-line and detects attack patterns.

Attack Detection

Attack	Example
SQL Injection	' OR 1=1
XSS	<script>alert(1)</script>
Brute Force	repeated failed logins
DDoS	many requests from many IPs
Port Scan	nmap detected

How DDoS Detection Works

System:

1. Counts requests per IP
 2. Checks traffic threshold
 3. If many IPs exceed threshold:
 - DDoS alert triggered
-

Module 5 — SecureReports

Purpose

Secure report upload system.

User can:

- Upload files
 - Store security reports
 - View uploaded reports
-

Working

1. User selects file
 2. Multer uploads it
 3. File stored in uploads folder
 4. Metadata saved into MySQL
-

Module 6 — Analyst Portal

Purpose

Admin monitoring dashboard.

Accessible only to admins.

Features

Activity Timeline

Shows:

- Threat scans
- Report uploads
- Log analyses

with timestamps.

Uploaded Reports Table

Shows all uploaded reports.

Live Cyber News

Fetches using:

NewsData.io API

Demonstrates:

External API integration

20. Protected Routes

Some pages require login.

Example:

/dashboard

/analyst-portal

If not logged in:

→ redirected to login page.

21. REST APIs Used

Method	Endpoint	Purpose
POST	/api/auth/register	Register
POST	/api/auth/login	Login
POST	/api/analyze	Threat analysis
POST	/api/logs	Log analysis
POST	/api/reports	Upload report
GET	/api/reports	Fetch reports
POST	/api/audit	Save activity
GET	/api/audit	Fetch logs
GET	/api/dashboard/stats	Dashboard stats

22. Deployment Concept

Currently:

Frontend → localhost:5173

Backend → localhost:5000

After deployment:

Frontend → Vercel

Backend → Render

Database → Railway / PlanetScale / MySQL server

23. Folder Structure

CyberSec-Nexus/

|

```
└── backend/
    ├── routes/
    ├── uploads/
    ├── db.js
    ├── server.js
    |
└── client/
    ├── src/
    |   ├── pages/
    |   ├── components/
    |   └── App.jsx
    |
└── package.json
```

24. AWT Concepts Successfully Demonstrated

AWT Concept	Used
SPA Architecture	
Frontend Framework	 React
Backend APIs	 Express
Database Connectivity	 MySQL
Authentication	
REST APIs	
File Upload	
External APIs	
Responsive UI	
Role-Based Access	
Dynamic Routing	

AWT Concept	Used
-------------	------

State Management	✓
------------------	---

HTTP Requests	✓
---------------	---

CRUD Operations	✓
-----------------	---

25. Major Learning Outcomes

Through this project you learned:

- Full-stack architecture
- Frontend-backend communication
- API creation
- Database operations
- Authentication systems
- Cybersecurity detection logic
- File handling
- Deployment basics
- React component structure
- REST architecture

26. Final One-Line Project Definition

“CyberSec Nexus is a full-stack cybersecurity web platform built using React, Node.js, Express, and MySQL that demonstrates authentication, threat analysis, log monitoring, secure file management, and admin monitoring using modern AWT technologies.”

Complete Technologies Explanation Used in CyberSec Nexus

1. React.js

Introduction

React.js is a JavaScript library used to build modern websites and web applications.

Instead of making one huge webpage, React divides the website into small reusable parts called:

Components

Example:

- Navbar
- Sidebar
- Dashboard card
- Login form
- Threat analysis panel

Each part becomes independent and reusable.

Why React is Popular

Because React:

- makes UI fast,
 - avoids page reloads,
 - organizes code properly,
 - allows reusable components,
 - is easier to maintain.
-

Where We Used React in Project

React was used for the ENTIRE frontend.

Everything visible in browser was made using React:

- Home page
- Login page
- Register page
- Dashboard

- ThreatLens
- LogSentry
- SecureReports
- AnalystPortal

React Files Used

File	Purpose
App.jsx	Main routing
main.jsx	Starts React app
Dashboard.jsx	Dashboard page
ThreatLens.jsx	Threat scanner
LogSentry.jsx	Log analyzer
Login.jsx	Login page
Register.jsx	Register page

Example in Project

```
function Dashboard() {  
  return (  
    <div>  
      <h1>Dashboard</h1>  
    </div>  
  )  
}
```

This creates a reusable Dashboard component.

Real-Life Analogy

React is like building a house using LEGO blocks.

Instead of making whole house at once:

- one block = Navbar

- one block = Sidebar
- one block = Card

Then combine them together.

2. Vite

Introduction

Vite is a frontend development tool.

It creates:

- very fast frontend server,
 - fast code compilation,
 - instant browser updates.
-

Why We Used Vite

Without Vite:

- React projects become slow during development.

With Vite:

- frontend loads almost instantly.
-

Where Used in Project

Frontend runs using:

`npm run dev`

This starts Vite server on:

`localhost:5173`

Files Related to Vite

File	Purpose
<code>vite.config.js</code>	Vite configuration
<code>package.json</code>	Contains Vite dependencies

Real-Life Analogy

Vite is like a turbo engine for frontend development.

3. React Router DOM

Introduction

React Router DOM handles navigation between pages.

Without React Router:

- every click reloads whole website.

With React Router:

- page changes instantly.
-

Where Used in Project

Used inside:

App.jsx

Example

```
<Route path="/dashboard" element={<Dashboard />} />
```

Meaning:

- if URL becomes /dashboard
 - show Dashboard component
-

Routes Used

Route	Page
/	Home
/login	Login
/register	Register
/dashboard	Dashboard
/threatlens	ThreatLens
/logsentry	LogSentry
/secure-reports	SecureReports

Route	Page
/analyst-portal	AnalystPortal

Real-Life Analogy

React Router is like GPS navigation inside the website.

4. Tailwind CSS

Introduction

Tailwind CSS is a styling framework.

Used to:

- add colors,
 - spacing,
 - shadows,
 - animations,
 - layouts.
-

Traditional CSS Problem

Normally:

background-color: blue;

padding: 20px;

border-radius: 10px;

Huge CSS files become messy.

Tailwind Solution

className="bg-blue-500 p-5 rounded-xl"

Everything written directly.

Where Used in Project

Used in EVERY frontend page.

Examples:

- buttons,
 - cards,
 - dashboard,
 - glowing effects,
 - risk bars,
 - layouts.
-

Example From Project

```
<button className="btn-glow px-6 py-4 rounded-xl">
```

Creates:

- glowing button,
 - padding,
 - rounded corners.
-

Real-Life Analogy

Tailwind is like ready-made fashion accessories for UI.

5. Axios

Introduction

Axios is used to send requests between frontend and backend.

Frontend cannot directly access database.

So:

Frontend → Axios → Backend → Database

Where Used in Project

Used in:

- Login
- Register
- ThreatLens
- LogSentry

- Dashboard
 - AnalystPortal
 - SecureReports
-

Example

```
axios.post("/api/analyze", { input })
```

Meaning:

- send user input to backend analysis route.
-

Types of Requests Used

Method Purpose

GET Fetch data

POST Send data

Real-Life Analogy

Axios is like a courier service carrying data between frontend and backend.

6. React Hooks (useState & useEffect)

Introduction

Hooks are special React functions.

useState

Stores changing data.

Example:

- username,
 - result,
 - loading,
 - reports.
-

Example

```
const [loading, setLoading] = useState(false)
```

useEffect

Runs code automatically.

Used for:

- fetching dashboard stats,
 - loading reports,
 - loading news.
-

Example

```
useEffect(() => {  
  fetchData()  
}, [])
```

Runs once when page loads.

Real-Life Analogy

- useState = memory box
 - useEffect = automatic machine
-

7. localStorage

Introduction

localStorage is browser memory.

Stores data even after refresh.

Where Used

Stores:

- login status,
 - username,
 - role.
-

Example

```
localStorage.setItem("isLoggedIn", true)
```

Why Important

Without localStorage:

- user gets logged out after refresh.
-

Real-Life Analogy

Like saving settings in mobile apps.

8. Node.js

Introduction

Node.js allows JavaScript to run on servers.

Normally JavaScript only runs inside browser.

Node.js allows:

- APIs,
 - databases,
 - backend logic.
-

Where Used

Entire backend.

Backend Responsibilities

- login checking,
 - database queries,
 - threat analysis,
 - log analysis,
 - uploads,
 - APIs.
-

Real-Life Analogy

Node.js is the engine room of the application.

9. Express.js

Introduction

Express.js is backend framework built on Node.js.

Makes API creation easy.

Where Used

Inside:

- server.js
 - all routes folder files.
-

Example

```
router.post("/login")
```

Creates login API.

APIs Created

API	Purpose
/api/auth/register	Register
/api/auth/login	Login
/api/analyze	Threat scan
/api/logs	Log analysis
/api/reports	File uploads

Real-Life Analogy

Express acts like traffic controller for requests.

10. MySQL

Introduction

MySQL is relational database.

Stores data permanently.

Tables Used

Table	Stores
users	User accounts
reports	Uploaded files
audit_logs	Activity history

Example

When user registers:

name → users table

email → users table

password → users table

Where Used

Connected inside:

db.js

Real-Life Analogy

MySQL is digital storage warehouse.

11. mysql2

Introduction

mysql2 is Node.js library for connecting backend to MySQL database.

Why Needed

Node.js alone cannot talk to MySQL.

mysql2 creates connection.

Example

```
db.query("SELECT * FROM users")
```

Real-Life Analogy

mysql2 is translator between Node.js and MySQL.

12. bcryptjs

Introduction

bcryptjs encrypts passwords.

Why Important

Never store passwords directly.

Bad:

hello123

Good:

\$2a\$10\$hKJ82...

Where Used

Inside:

auth.js

During Register

Password hashed before saving.

During Login

Entered password compared with hashed password.

Real-Life Analogy

Like converting password into secret code.

13. Multer

Introduction

Multer handles file uploads.

Where Used

SecureReports module.

What It Does

When user uploads file:

- receives file,
 - stores file,
 - generates filename,
 - saves metadata.
-

Example

```
upload.single("file")
```

Real-Life Analogy

Multer acts like file receptionist.

14. Nodemailer

Introduction

Nodemailer sends emails using Node.js.

Where Used

User registration alerts.

What Happens

When new user registers:

- email automatically sent to admins.
-

Why Added

Demonstrates:

- email service integration,
 - backend automation.
-

Real-Life Analogy

Like automatic notification system.

15. CORS

Introduction

CORS means:

Cross-Origin Resource Sharing

Problem

Frontend:

localhost:5173

Backend:

localhost:5000

Browser treats them as different origins.

Without CORS:

- communication blocked.
-

Solution

Enable CORS in backend.

Example

```
app.use(cors())
```

Real-Life Analogy

CORS is permission gate between frontend and backend.

16. REST API

Introduction

REST API is communication system between frontend and backend.

Example Flow

User clicks Analyze

↓

Frontend sends POST request

↓

Backend processes

↓

Backend sends response

↓

Frontend displays result

API Methods Used

Method Purpose

GET Fetch

POST Send

Real-Life Analogy

REST API is conversation language between frontend and backend.

17. NewsData.io API

Introduction

External API for cyber news.

Where Used

Analyst Portal.

What It Does

Fetches:

- latest cybersecurity news,
 - attack reports,
 - cyber incidents.
-

Example

<https://newsdata.io/api/1/news>

Why Important

Demonstrates:

Third-party API integration

18. Protected Routes

Introduction

Some pages should only open after login.

Where Used

Dashboard and Analyst Portal.

Working

If:

`isLoggedIn = false`

Redirect to:

`/login`

Real-Life Analogy

Like restricted rooms needing ID card access.

19. Environment Variables (.env)

Introduction

Stores secret values separately.

Example

VITE_API_URL=http://localhost:5000

Why Important

Avoids hardcoding URLs everywhere.

Real-Life Analogy

Like central settings panel.

20. Git & GitHub

Git Introduction

Version control system.

Tracks code changes.

GitHub Introduction

Cloud platform for storing Git projects online.

Where Used

Project uploaded to GitHub repository.

Commands Used

git init

git add .


```
git commit -m "Initial upload"
```

```
git push
```

Why Important

- backup,
 - collaboration,
 - deployment,
 - project sharing.
-

21. Deployment Platforms

Vercel

Used for frontend deployment.

Render

Used for backend deployment.

Purpose

Convert:

localhost project

into:

live website

accessible from internet.

22. Cybersecurity Concepts Used

Concept	Where Used
Phishing Detection	ThreatLens
SQL Injection Detection	LogSentry
XSS Detection	LogSentry
DDoS Detection	LogSentry

Concept	Where Used
Brute Force Detection	LogSentry
Password Hashing	Authentication
Role-Based Access	Analyst Portal
Secure File Upload	SecureReports

23. Most Important Reality About Project

Your project is:

Full-Stack Rule-Based Cybersecurity Platform

NOT:

- AI system
- Machine learning engine
- Real antivirus

It demonstrates:

- cybersecurity concepts,
- attack detection logic,
- full-stack web development,
- AWT technologies.

That is already strong enough for a mini-project.